

**AIM-** To interface a push button as digital input and demonstrate LED control by toggling its state on each valid button press.

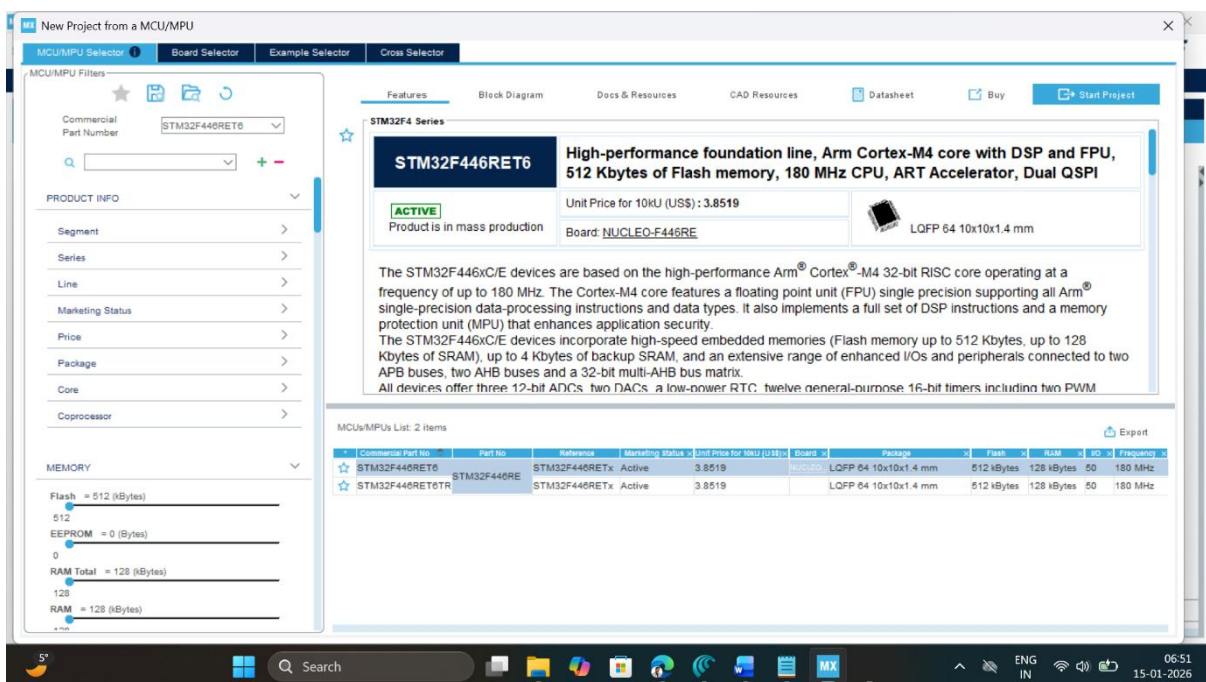
**APPARATUS-** STM32 Nucleo-F446RE development board, USB Type-A to Mini-B cable, STM32 CUBE IDE, STM32 CubeMX.

## THEORY

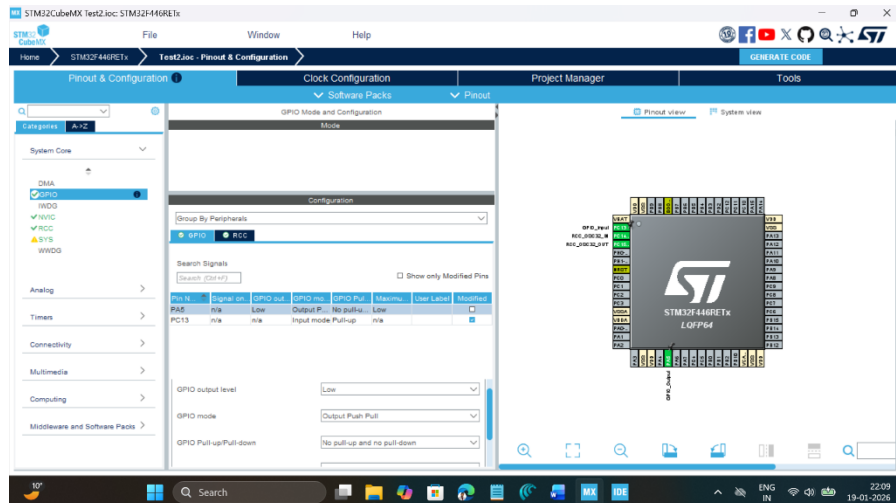
STM32F446RE provides multiple general-purpose I/O (GPIO) ports (GPIOA–GPIOH) that can be configured as input, output, alternate function, or analog through GPIO control registers or HAL. On the Nucleo-F446RE board, the green user LED **LD2** is connected to Arduino D13, which corresponds to microcontroller pin **PA5** on most board revisions. When PA5 is configured as a **push-pull output**, the pin can be driven to logic high (near 3.3 V) or logic low (0 V), which in turn turns the LED on and off through an on-board current-limiting. Using STM32CubeIDE and HAL, functions like HAL\_GPIO\_TogglePin() and HAL\_Delay() can be used inside an infinite loop to generate a periodic LED blink without directly programming registers.

## PROCEDURE

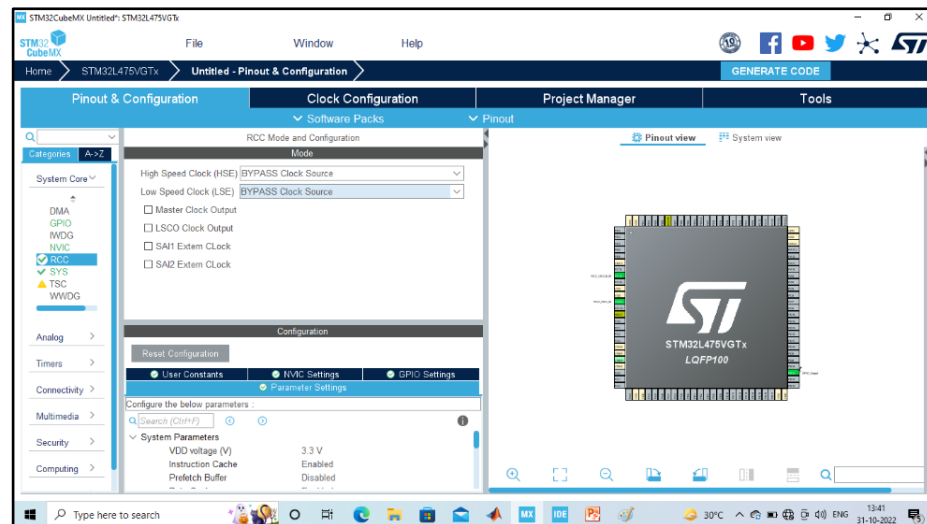
1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.



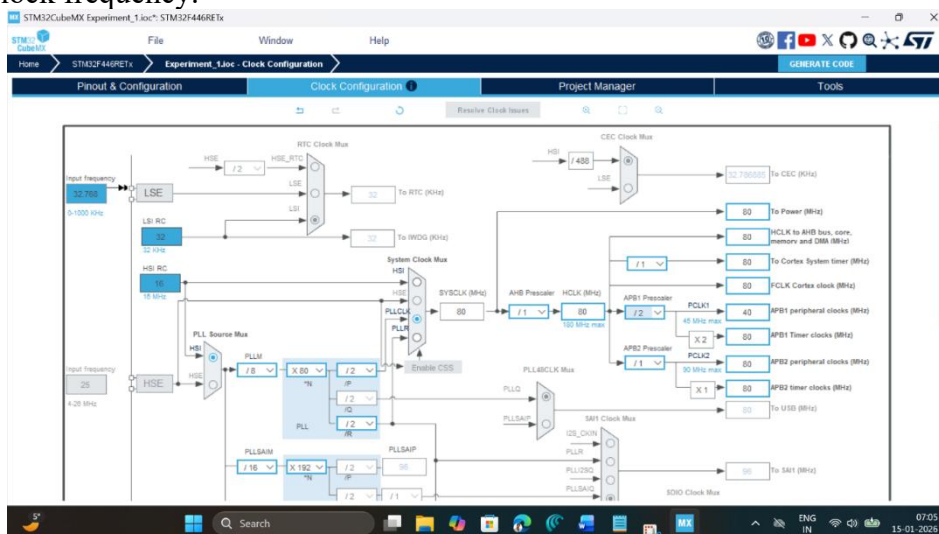
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L446RE.
3. In system Core select GPIO and right click on pin PA5 to make it as GPIO output pin as shown below as LED (Inbuilt LED) and PC13 as Pushbutton (USER) relates to it in the board.



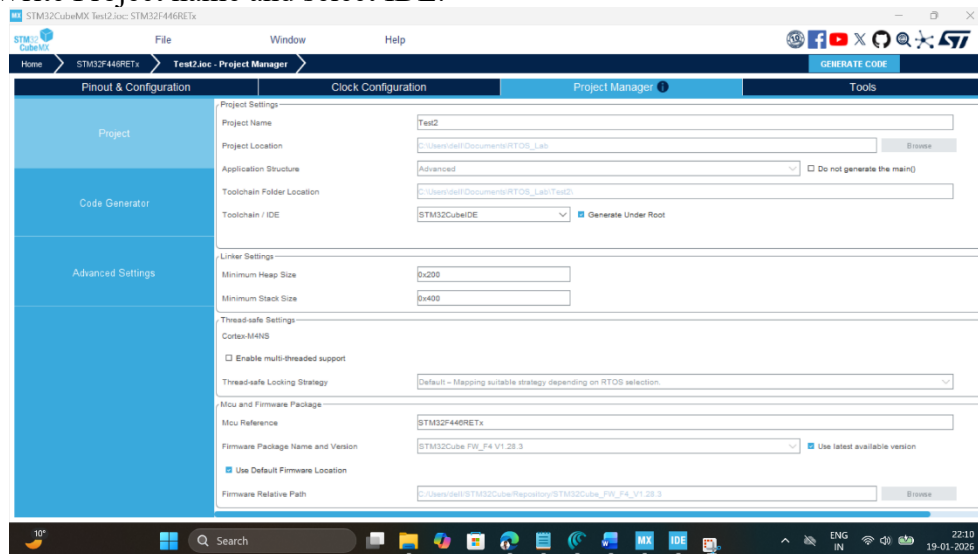
4. **Clock Selection:** Select the internal clock by clicking RCC and selecting BYPASS clock source



5. **Clock Configuration:** Configure clock as shown below to get maximum internal clock frequency.



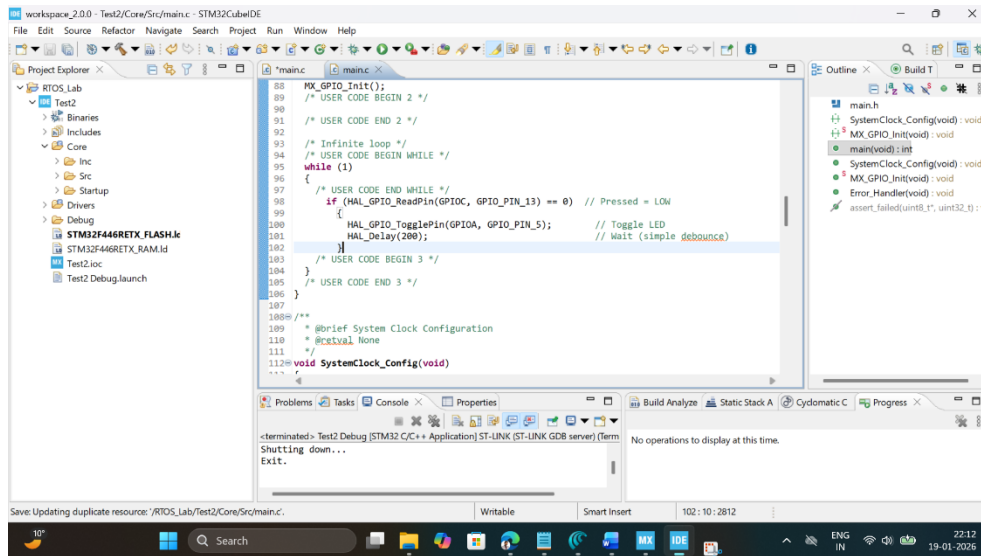
## 6. Write Project name and select IDE.



7. Click on generate code.
8. Click on open project.
9. Click 'OK' on the generated popup showing "successfully imported the project on the workspace" and the project will be successfully added to the cube IDE.
10. Open the main source code on Cube IDE by clicking on **main.c**
11. To generate bin file and hex file, right click on project and click properties.
12. Expand C/C++ Build, click on setting, click on MCU post build outputs and select convert to binary file as well as convert to hex file and click "Apply and Close".
13. After configuration write only below two instructions in the user while loop and compile the program and ensure there will be zero error after compiling.

```

while (1)
{
    // Read onboard button (PC13)
    if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == 0) // Pressed = LOW
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);    // Toggle LED
        HAL_Delay(200);                          // Wait (simple debounce)
    }
}
  
```



14. To run this program, connect the board with USB cable, click on build, click 'ok' and click 'switch' on popups.

15. Now you are in debugging mode, click 'Resume' to execute the program.

## OBSERVATION:

### Button Press vs LED State Observation

| Trial No. | Button Action      | Button Pin State (PC13) | LED Pin State (PA5) Before | LED Pin State (PA5) After | Observation                                                |
|-----------|--------------------|-------------------------|----------------------------|---------------------------|------------------------------------------------------------|
| 1         | Initial (No press) | 1 (HIGH)                | 0 (OFF)                    | 0 (OFF)                   | LED remains OFF when button not pressed                    |
| 1         | 1st Press          | 0 (LOW)                 | 0 (OFF)                    | 1 (ON)                    | LED toggles ON on first press                              |
| 2         | Release            | 1 (HIGH)                | 1 (ON)                     | 1 (ON)                    | LED state remains unchanged on release                     |
| 3         | 2nd Press          | 0 (LOW)                 | 1 (ON)                     | 0 (OFF)                   | LED toggles OFF on second press                            |
| 4         | 3rd Press          | 0 (LOW)                 | 0 (OFF)                    | 1 (ON)                    | LED toggles ON again                                       |
| 5         | Rapid Press        | 0 (LOW)                 | Depends on last state      | Opposite of last state    | LED toggles once per valid press due to delay (debouncing) |

## RESULT

For all button presses, the LED toggled once and the state remains same after the release of button, and this can be verified by values filled in observation table.

## Reflection Summary

This experiment helped me understand how a microcontroller can interact with real-world devices using digital input and output pins. In this setup, a push button was used as an input device and an LED was used as an output device. When the button was pressed, the microcontroller detected the change in logic level and performed the operation of toggling the LED. This clearly showed how a simple electrical signal can be processed through programming to control an external component. One of the key learnings from this experiment was the importance of proper input handling. The push button operates as an active-low input, meaning it gives a LOW signal when pressed.

Understanding this behavior is necessary for writing correct logic in the program. Another important concept learned was switch bouncing. Due to mechanical imperfections, a button may generate multiple transitions during a single press. By introducing a delay in the program, the system was able to ignore unwanted fluctuations and respond only once per press, making the operation stable and predictable.

This experiment also introduced the practical use of STM32CubeMX and HAL libraries for GPIO configuration and control. Instead of directly writing register-level code, HAL functions simplified the programming process and made the code easier to maintain. This allowed focus on logic design rather than hardware complexity.

In conclusion, this experiment strengthened my basic knowledge of embedded systems by showing the relationship between hardware components and software instructions. The skills gained from this experiment will be helpful in future applications such as keypad interfacing, sensor-based systems, and automated control projects. It also highlighted the importance of testing and observing system behavior before moving to more advanced designs.